

MSc Project Report in Computer Science - October 2026

Year: 2025-26

Syed Mohiuddin
Mudassir

415789

JOBREC

AI-POWERED JOB RECOMMENDATION SYSTEM

Design, Development and Evaluation of a Hybrid Machine Learning-Based Job Recommendation System

MSc Computer Science Project Report

Submitted by

Submitted to

11/10/2026

Abstract

This report documents the design, implementation and offline evaluation of JobRec, a web-based job recommendation prototype that matches data-related vacancies to profiles containing skills, experience and preferred roles. The solution combines content-based recommendation, implemented through TF-IDF text representations and explicit skill-set similarity, with a collaborative model based on weighted user–job interactions, item similarity and TruncatedSVD. A weighted hybrid integrates the two signals while falling back to content scoring for new users. The implementation uses Python, scikit-learn, Flask and SQLite and includes profile editing, keyword browsing, recommendations, saving, interaction tracking, job details and machine learning evaluation pages. The curated catalogue contains 8,000 public job postings and 127 distinct annotated skills. Offline evaluation on 300 simulated users and 4,213 generated interactions compares four ranking methods at five, ten and twenty suggestions. At k=10, the hybrid records precision 0.089, recall 0.226, hit rate 0.480 and NDCG 0.182, outperforming the included single-method baselines under the defined synthetic protocol. The report interprets these results alongside important limitations: interactions are simulated, job postings are historical, salary information is incomplete and the hybrid coefficient was selected within the reported evaluation environment. The study therefore establishes an inspectable, evaluated prototype, not a validated predictor of recruitment outcomes.

Contents

Chapter 1: Introduction 4

Chapter 2: Literature Review and Technical Foundations 6

Chapter 3: Requirements Analysis and Project Planning 10

Chapter 4: Dataset Preparation and Exploratory Analysis 12

Chapter 5: Recommendation Methodology 16

Chapter 6: System Architecture and Implementation 19

Chapter 7: Experimental Design and Evaluation Protocol 22

Chapter 8: Results and Interpretation 24

Chapter 9: Testing, Ethics and Critical Discussion 27

Chapter 10: Recommendations for Further Development 29

Chapter 11: Conclusion 30

References 31

Appendix A: Source Artefacts and Traceability 31

Chapter 1: Introduction

1.1 Background and motivation

Finding a suitable job has become a problem of navigating information as much as a problem of identifying available vacancies. Online services publish large numbers of advertisements, but the volume of results does not necessarily make it easier for an individual to recognise which roles genuinely fit their capabilities. A search for “data analyst,” for example, may return positions ranging from reporting and dashboard development to engineering, healthcare analytics and financial modelling. These titles overlap, yet the practical skills expected by employers can be quite different. A person who knows SQL, Excel and Tableau may be qualified for several of these opportunities while lacking some requirements of others. Reading every advertisement and comparing it manually is costly in time and attention.

Conventional keyword search is useful when a user already knows which words to enter. Its principal limitation is that it treats those words as the main expression of intent. A candidate's skills, related experience and previous choices are richer signals. Someone interested in business intelligence might overlook an otherwise appropriate advertisement that uses “reporting analyst” instead of “business intelligence analyst.” Conversely, the presence of “Python” in a long job description does not establish that Python is central to the post or that the remaining requirements are suitable. Effective job matching therefore calls for a ranking process that combines explicit profile information with evidence from the candidate's interactions. This is the practical problem addressed by JobRec.

Recommender systems organise large collections of items around the interests of individuals rather than relying only on a generic ordering. The underlying idea is established in research on information filtering, personalisation and collaborative behaviour (Resnick & Varian, 1997; Adomavicius & Tuzhilin, 2005). Applying this approach to recruitment offers a useful extension: the items are job advertisements, while the features include occupation titles, descriptions, required skills, work location and employment conditions. JobRec implements this idea as a browser-accessible prototype centred on the data-analytics labour market. Its central design decision is to combine information derived from job text with patterns of behaviour represented by user–job interactions.

1.2 Problem statement

The problem can be expressed as follows: given a catalogue of job postings and a profile containing a candidate's skills, experience and desired role, how can a system order the postings so that more relevant opportunities appear near the top? This seemingly simple question includes several distinct challenges. Job advertisements are free-text documents that vary in terminology and detail. Users may describe the same skill in different ways, such as “Power BI,” “PowerBI”

or “power_bi.” A new user may have a complete skills profile but no recorded behaviour. Another user may have browsing history that reveals interests not stated explicitly in the profile. Some advertisements contain salary data, while most in the supplied sample do not. Finally, there must be a defensible way to assess whether an ordering is an improvement rather than merely an attractive list.

JobRec addresses these challenges in stages. Text and annotated skill fields are transformed into comparable mathematical representations. The content component scores how well a profile corresponds to each posting. A collaborative component learns from simulated views, saves and applications, assigning larger weights to interactions assumed to indicate stronger interest. A hybrid score blends the two signals. The web interface then provides ranked suggestions, highlights required skills already held by the user, and identifies additional skills in each vacancy. This last feature is especially relevant in a career-development setting: a recommendation is more informative when a candidate can see why it appeared and where the gaps lie.

1.3 Aim, objectives and research question

The overall aim of the project is to design, implement and evaluate an explainable, skills-oriented job recommendation system using a hybrid of content-based and collaborative filtering. The main research question is: **How effectively does combining job-description similarity and interaction-based preferences improve ranking performance over either approach alone in the supplied JobRec evaluation environment?** This wording is deliberately tied to the available evidence. It does not assume that the system has already improved real-world hiring outcomes or that its simulated users are representative of actual applicants.

The project's operational objectives are to prepare a consistent job dataset; represent vacancies and candidate profiles using normalised skills and text features; create independent content-based and collaborative scoring methods; blend the methods while supporting users with no interaction history; deliver the rankings through a functioning Flask application; and compare the ranking methods using established top-k retrieval measures. Further objectives are to provide intelligible explanations, support profile-based filters, record user activity and preserve a reproducible set of evaluation outputs.

The project artefacts provide evidence for these objectives. The archive contains the curated CSV, Python modules implementing the recommendation methods, a Flask application with HTML templates, model-training and evaluation scripts, a saved model, structured results, a plot and screenshots of the interface. The presence of those files demonstrates an implemented prototype and a recorded offline experiment. It does not, by itself, establish performance under production traffic, fairness in actual recruitment or the satisfaction of real users. Maintaining that distinction is part of sound project reporting.

1.4 Scope and contribution

The study concentrates on analytical and technical vacancies contained in a fixed catalogue of 8,000 postings. It considers skill overlap, textual relevance and user–job interaction patterns. It includes features for browsing, saving, viewing details and registering an action labelled "Apply." In the current code, that action is saved as an interaction in the local database; it is not an integration that submits an application to an employer. The site is therefore best described as a recommendation and interaction-tracking prototype rather than a complete recruitment marketplace.

The principal contribution is not a newly invented machine learning algorithm. It is the integration of recognised methods into a coherent, inspectable application, together with a comparative evaluation and practical explanations. That distinction matters academically: a useful software engineering contribution can involve selecting methods appropriate to a problem, connecting them correctly, documenting design trade-offs and examining limitations. Here, the combination of TF-IDF, explicit skill similarity, item-based collaborative filtering and dimensionality reduction creates several opportunities to analyse how different information sources affect recommendations.

A second contribution is the explicit treatment of the cold-start case. When the system cannot obtain a meaningful collaborative signal for a user, it returns to content-only scoring. This is valuable because new graduates may provide skills before they have generated any platform history. Finally, the interface displays matched and missing skills rather than relying solely on a numerical score. A candidate can evaluate the recommendation in understandable terms, even if the score itself is not a probability of employment.

Chapter 2: Literature Review and Technical Foundations

2.1 Recommendation as an information-filtering problem

Recommender systems emerged in response to the difficulty of finding individually useful items in expanding digital collections. Rather than asking all users to browse the same catalogue in the same order, a recommender attempts to infer which items deserve the user's attention. Resnick and Varian (1997) characterised recommendation as a means of sharing preferences and filtering choices. Later surveys distinguished approaches based on item properties, community behaviour and combinations of methods (Adomavicius & Tuzhilin, 2005). These distinctions are useful when designing a job recommender because the information required by each approach is quite different.

In content-based recommendation, the system asks whether the attributes of an item match a representation of the user. In a job context, these attributes include occupation vocabulary,

programming languages, software packages and written responsibilities. Collaborative filtering instead asks which items tend to be selected by users with related behaviour. A hybrid method uses more than one of these information sources. None is universally superior. Their comparative usefulness depends on catalogue composition, available user history, sparsity and the definition of relevance (Ricci et al., 2022).

2.2 Content-based filtering and text representation

A content-based approach is appealing for job recommendations because a new advertisement generally arrives with descriptive information. Even in the absence of interaction logs, its title, skills and description can be analysed. The difficulty lies in converting natural language into a representation that can be compared consistently. Term frequency–inverse document frequency, commonly abbreviated TF-IDF, is a classical method that weights words by their importance within a document relative to the wider collection. Salton and Buckley (1988) showed why the choice of term weighting matters in information retrieval. Words occurring repeatedly in one description may carry useful meaning, while words occurring in almost all descriptions contribute relatively little to distinguishing posts.

For example, “data” may occur across much of an analytics dataset, while an unusual technology name may distinguish a more specific role. TF-IDF encodes such differences numerically. After vectorisation, cosine similarity measures the angle between a user's feature vector and a posting vector. A larger cosine indicates a closer direction in the feature space. The measure is insensitive to the absolute length of vectors after normalisation, which is helpful when the profile is much shorter than a full advertisement. However, lexical similarity should not be confused with semantic understanding. Related expressions can remain distant when they do not share words or recognised tokens.

JobRec attempts to lessen this problem by using an additional binary skill representation. Explicitly annotated skills are useful because they provide a more structured statement of requirements than the description alone. The project combines text similarity and skill-vector similarity, giving each half the content score when a usable profile skill vector is available. This is a practical way to ensure that a mention of a relevant technology influences the ranking, although it also makes ranking quality dependent on the accuracy and coverage of the extracted skill labels.

2.3 Collaborative filtering and implicit feedback

Collaborative filtering derives recommendations from relationships among users and items. The familiar intuition is that people with similar choices may find similar items valuable. Explicit ratings are uncommon in job platforms, but many possible interactions are recorded naturally: opening a posting, bookmarking it or indicating an intention to apply. Such behaviour is called

implicit feedback because the action is evidence of possible interest rather than a direct numerical statement of preference. Hu et al. (2008) explained how implicit observations can support recommendation while also creating uncertainty about the strength and meaning of a signal.

The JobRec design weights a view as 1, a save as 3 and an apply action as 5. The order is intuitively plausible: saving usually indicates more commitment than briefly opening a listing. Yet these values are design assumptions, not experimentally validated measures of user preference. A saved vacancy can later prove unsuitable, and an application can be motivated by convenience rather than strong preference. The report therefore treats interaction weights as a controlled modelling choice rather than a psychological fact.

Two collaborative strategies appear in the project. Item-neighbourhood filtering estimates associations between jobs from co-interaction patterns. Matrix factorisation provides a lower-dimensional account of these patterns, allowing latent similarities to affect scores even when exact job pairs have limited overlap. Koren et al. (2009) described the importance of latent factors in collaborative recommendation. JobRec uses truncated singular value decomposition rather than the same fully developed optimisation methods discussed in all matrix-factorisation literature, so it is accurate to describe its component specifically as TruncatedSVD-based latent scoring.

2.4 Why a hybrid model is appropriate

The strengths and weaknesses of content and collaborative approaches are complementary. Content methods can serve a user immediately after profile creation, but may recommend many near-identical positions and depend heavily on how requirements are written. Collaborative methods can reflect patterns that are not present in a job description, but suffer when few people have interacted with a listing or the user has not yet created history. Burke (2002) documented different ways of combining recommender methods, including weighted hybrids. A weighted blend is attractive for a constrained prototype because it is simple to implement and inspect.

JobRec selects a final score as a weighted combination of the content and collaborative signals. The mixing coefficient alpha represents the relative importance assigned to content. A value of 0.3 means that, after the system's score normalisation, 30% of the combined score comes from content and 70% from collaborative scoring. These percentages refer to a formula, not to calibrated probabilities or measured causal contributions. The reported evaluation compares several alpha values and finds 0.3 strongest at k=10 on the supplied synthetic test set.

2.5 Job recommendation in prior research

Job recommendations involve specialised information that extends beyond generic product suggestions. Paparrizos et al. (2011), for instance, studied machine-learned job recommendation

using historical career transitions and employee-related information. Their work shows that past professional movement can be informative, but it also illustrates a broader point: the definition of an appropriate career move depends on available user and employer data. JobRec takes a different and more accessible route by building from public job descriptions and mock interactions rather than attempting to predict a person's next institution.

This distinction affects what can be claimed. JobRec is primarily a vacancy-ranking system for a self-described profile. It cannot determine employer acceptance, professional licensing, immigration eligibility, qualifications that were not entered, or whether an advertisement remains open. Those matters may be represented imperfectly or not at all in the input data. A strong ranking on an offline relevance test is therefore one component of a complete employment-matching solution, not proof of successful placement.

2.6 Evaluation in recommendation research

Recommendation evaluation commonly asks whether relevant items occur near the top of an ordered list. Precision at k measures the proportion of displayed suggestions that are relevant; recall at k measures the share of all relevant items retrieved; and hit rate asks whether at least one relevant item appears. A ranking-aware measure, normalised discounted cumulative gain (NDCG), rewards relevant items more when they appear earlier. Järvelin and Kekäläinen (2002) developed the cumulated-gain framework that underpins this type of ranking evaluation.

These measures capture different priorities. A user examining only five vacancies may care particularly about precision@5; an analyst assessing how broadly the model reaches relevant openings might examine recall@20. NDCG is valuable when the difference between placing a relevant job first and placing it tenth matters. However, all such measures depend on the availability of reliable relevance judgements. In the current project, held-out synthetic interactions provide those judgements. That allows repeatable algorithm comparison, but it does not provide the same evidence as judged relevance from genuine job seekers.

2.7 Responsible use, explanation and limitations of automation

Career-related recommendations can shape opportunities, so transparency matters. A system may repeatedly surface common roles and exclude less frequent ones, or reinforce the behaviour of users already dominant in the data. Research on fairness in recommender systems emphasises that performance must be considered in relation to the people affected, the items receiving exposure and the context of deployment (Ricci et al., 2022). JobRec's explanation labels are a helpful first step because they make a concrete skill-level reason visible to the user.

Nevertheless, an explanation based on matched skills does not show every contribution to a hybrid score. Collaborative signals may place a job high even when the user's profile has limited direct skill overlap. Likewise, a high score does not imply that the system assessed equal

opportunity, protected attributes or the authenticity of an advertisement. A responsible interpretation is that the interface supplies suggestions and supporting evidence, leaving the final decision with the job seeker. Before use with real applicants, privacy consent, fairness checks, job freshness and the consequences of inaccurate ranking would require more systematic attention.

Chapter 3: Requirements Analysis and Project Planning

3.1 Identifying the system's users

The primary intended user is a person looking for relevant employment opportunities, especially within analytics-related occupations. This person needs to state what they know, which role they would like and any practical restrictions on work location. The secondary user is a researcher, developer or supervisor examining whether the recommendation method works as intended. This second perspective explains why the application includes a public evaluation page, architectural explanation and access to measurable results. These two perspectives need to remain distinct: the ordinary candidate should not have to interpret machine learning formulas, while the evaluator must be able to inspect the decision process.

A further stakeholder is an administrator or maintainer who loads data, trains models and handles errors. The current prototype supplies command-line scripts for those activities but does not implement a dedicated administrative dashboard. Recruitment staff and employers are outside the application's directly implemented workflow. That omission is important when describing the system boundary: the prototype supports candidate-side exploration, not end-to-end recruitment management.

3.2 Functional requirements

The first functional requirement is profile creation. A user must be able to register, authenticate and enter a name, education, skills, preferred job title and experience summary. The system must normalise the entered skills before using them for ranking. The profile also allows location preferences, a remote-only restriction and an optional minimum annual salary. The recommendation mechanism must then produce a ranked list containing identifiers, titles, employers, work arrangements, the overall model score and a breakdown of the matched and missing skills.

A second set of requirements concerns ordinary browsing and interaction. Users should be able to search postings by words or phrases, inspect a job description, view comparable jobs and save a position for later review. Clicking a vacancy records a view when the visitor is authenticated. Saving and selecting Apply record distinct interaction types, allowing the collaborative

component to update from activity. A history page must display saved and applied vacancies so that the user can recognise what has been tracked.

The application must also accommodate two edge cases: a new profile without previous interactions and a requested filter that excludes many vacancies. The first should not prevent a recommendation list from being generated; the content-based component can still operate. The second should not silently ignore a filter. Ideally the interface should tell a user when too few eligible matches remain, rather than presenting unfiltered results as though they satisfied the user's constraints. The supplied implementation applies filters before constructing the list, although its handling of absent salary values needs careful interpretation.

For research and integration, a further requirement is to make evaluation outputs reproducible and inspectable. The project contains a script that saves results in JSON and Markdown as well as a graph. The Flask site also exposes an API for recommendations and individual job details and a health endpoint. These routes support demonstrations and basic integration, but the current project does not document formal service-level guarantees.

3.3 Non-functional requirements

Usability is particularly important because recommending suitable vacancies should reduce effort, not create another complex interface. The visible design uses a short navigation bar, structured profile fields, concise ranking cards and direct actions for viewing, saving and recording applications. The matched-versus-missing skill display is meaningful because it translates a complex ranking into a comparison that many users can understand without specialised technical knowledge.

Performance is another requirement. The curated dataset is intentionally limited to 8,000 postings and the model uses sparse matrices for much of its representation. That supports local experimentation on commonly available hardware. The project does not supply benchmarked request latency or concurrent-user throughput, so any claim of production-scale performance would require new measurements. Maintainability is addressed through separate Python modules for loading data, content scoring, collaboration, simulation, hybrid ranking and evaluation. This separation makes individual components easier to read and change.

Security and privacy are expected requirements for any system accepting account information. Password hashing is present through Werkzeug, and SQLite persists profiles and interactions. However, the use of a development fallback for the Flask secret key and the absence of documented production security testing mean that the current setup should be regarded as demonstrational. A production adaptation would need more robust credential management, input validation, access controls, privacy documentation and deployment hardening.

3.4 Requirements mapped to project evidence

The source archive provides several kinds of evidence rather than relying on the website images alone. The CSV establishes the existence of a job catalogue. *jobrec/content_model.py*, *collab_model.py* and *hybrid.py* show how scoring is carried out. *app/app.py* and the HTML templates show that routes and interfaces are implemented. The database schema defines users and interactions. The stored evaluation files allow the reported figures to be checked without guessing from a chart. Finally, the training and evaluation scripts explain how the model and results are produced.

This traceability approach also reveals where the evidence is limited. The project archive contains no record of supervised user acceptance testing with real job seekers. The screenshots illustrate a working example, but do not prove accessibility compliance, resistance to malicious inputs or successful delivery of applications to outside organisations. A complete report should explicitly record these as boundaries rather than manufacture testing outcomes. With those limitations stated, the available materials are sufficient to document the implemented algorithm and its recorded offline comparison in detail.

Chapter 4: Dataset Preparation and Exploratory Analysis

4.1 Source and provenance

The project README identifies Luke Barousse's public Kaggle dataset, “Data Analyst Job Postings [Pay, Skills, Benefits],” as the source of the job advertisements (Barousse, n.d.). The dataset originates from publicly accessible job-search listings and includes structured fields alongside natural-language descriptions. The supplied repository contains a curated CSV rather than the original quarterly raw files. A preparation script explains how the curated file can be recreated if those source files are downloaded separately. Thus, the observed analysis concerns the 8,000-row curated file actually supplied, not every posting available on the original platform.

Provenance matters for two reasons. First, it allows a reader to distinguish public job information from simulated user data. Second, it sets geographical and historical boundaries for interpretation. The supplied postings are strongly associated with the United States and include historical advertising dates, not a live feed of currently vacant positions. The system can demonstrate ranking on those descriptions, but the presence of a job in the interface should not be interpreted as proof that an employer is still hiring for it.

4.2 Preparation pipeline

The data-preparation script describes a sequence of quality controls. It combines quarterly CSV files, removes rows lacking a title or description, and deduplicates on title, company and

description. It then converts a field containing annotated skills into a cleaned list, retains only advertisements with at least two identified skills and trims lengthy description strings to 1,500 characters. Work-from-home indicators are converted into Boolean values, annual salary figures are coerced into numeric form where possible, and the original timestamp is transformed into a posting date. If more than 8,000 rows remain, a deterministic random sample is drawn using seed 42.

The README records intermediate counts of 28,375 raw postings, 20,446 after deduplication and 13,142 after the minimum-skills condition. These counts are reported from project documentation; the ZIP does not contain the raw quarterly files with which to independently reconstruct them. The final count, however, was checked against the supplied CSV and is exactly 8,000. Each final job receives an integer identifier used to connect model rankings, job-detail routes and database interactions.

The choice to require at least two annotated skills is understandable because the project is specifically about skills-based ranking. It creates a coherent technical corpus in which most job pairs can be meaningfully compared by explicit skills. At the same time, this choice excludes advertisements with sparse annotation and can therefore change which employers and occupations are represented. It is a filtering decision, not a neutral reflection of the labour market.

4.3 Schema and observed data profile

The curated file contains 11 columns: *job_id*, *title*, *company*, *location*, *source*, *schedule_type*, *work_from_home*, *salary_yearly*, *posted_date*, *skills* and *description*. The *skills* field stores semicolon-separated identifiers, while the description contains readable posting text. During loading, a derived *skill_list* column splits the identifiers and a *doc* column combines repeated title terms, repeated skills and description text for vectorisation. These derived fields are model inputs, not additional source columns in the exported CSV.

Table 1. Observed features of the supplied job catalogue

Indicator	Value
Curated job records	8,000
Unique skill tokens	127
Distinct title strings	4,088
Distinct company strings	3,954
Remote-labelled records	3,474 (43.4%)
Records with salary	1,206 (15.1%)
Posting dates	Nov 2022–Sep 2023

Direct inspection of the CSV found 8,000 distinct job identifiers, 4,088 distinct job titles, 3,954 distinct employer-name values and 224 distinct location strings. The dataset includes 127 unique skill tokens. Approximately 3,474 postings are marked as work-from-home, equivalent to 43.4% of the sample, while 4,526 are not. These values describe only the curated sample. They should not be generalised to the overall proportion of remote opportunities in an economy or even in the original Kaggle collection.

The salary field is markedly incomplete. Only 1,206 of the 8,000 records contain a numerical annual salary, leaving 6,794 missing values. That is about 15.1% recorded and 84.9% missing. This is one of the most consequential data-quality observations because the interface offers a minimum-salary filter. The code currently converts missing salaries to positive infinity for the purpose of that comparison, meaning that a vacancy without an advertised salary can pass a minimum threshold. The filter therefore cannot be described as guaranteeing that every displayed vacancy pays at least the stated amount.

4.4 Skill distribution and vocabulary

The most frequently represented skills provide an overview of the corpus's orientation. SQL appears in 5,698 postings, Excel in 3,687, Python in 3,330, Tableau in 3,078 and Power BI in 2,854. R appears in 1,956 postings. Other identified technologies include SAS, Azure, AWS, Snowflake, Looker and spreadsheet-related tools. This concentration makes the collection suitable for studying recommendations to data analysts, but it also means that popular software names can dominate observable overlap. A highly specialised career path with less frequent terminology may receive fewer directly comparable listings.

Table 2. Ten most common annotated skills in the curated CSV

Skill	Postings	Skill	Postings
SQL	5698	Excel	3687
Python	3330	Tableau	3078
Power BI	2854	R	1956
SAS	992	Word	983
PowerPoint	924	Azure	690

Normalising skill names helps reduce avoidable mismatches. The project maps variants such as “PowerBI” to *power_bi*, “MS Excel” to *excel*, and “Amazon Web Services” to *aws*. It trims whitespace, converts text to lowercase and replaces remaining spaces with underscores. When the user enters skills separated by commas, semicolons, vertical bars or new lines, duplicates are removed. Such rules are modest but useful because a system should not treat several spellings of the same common tool as separate qualifications.

There are still limits to this normalisation. A fixed alias dictionary cannot cover every acronym, vendor name or emerging framework. It can also introduce inconsistency when a synonym normalises to a term that is absent from the dataset's own vocabulary. For example, the rules map “machine learning” to *ml*, while the interface examples contain *machine_learning* as a skill string. Resolving the vocabulary end to end, rather than only cleaning user input, would improve future robustness.

4.5 Location and historical coverage

The location field is not evenly distributed. “Anywhere” appears in 3,474 rows, “United States” in 2,600 and several US cities account for substantial portions of the remaining records. Although the data represent many distinct organisations and titles, these location labels reveal a geographically concentrated collection. The recorded posting dates in the supplied file range from November 2022 to September 2023. Accordingly, a report written later must avoid treating the postings as evidence of present-day hiring conditions.

4.6 Missingness, representativeness and quality risks

The data combine relatively complete textual fields with uneven structured metadata. Title, company, location, skill list and description are present in the supplied curated file; salary is sparse. The natural-language fields have already been truncated and normalised to some degree, which makes the prototype smaller and easier to process but also removes information that could have distinguished complex roles. A requirement mentioned after the first 1,500 characters may not contribute to the ranking, although it may have been present in the original listing.

Sampling and annotation introduce further uncertainty. Retaining posts with annotated skills can favour jobs that follow conventional terminology and can under-represent vacancies whose requirements are expressed indirectly. Duplicate removal reduces repetitive results but only catches records matching the chosen fields exactly. Multiple advertisements for materially identical roles can remain if the wording or company string differs. It would therefore be worthwhile in a future pipeline to measure near-duplicates and reporting coverage rather than assuming that cleaning has eliminated every data problem.

Finally, the dataset does not contain verified suitability labels from actual applicants or hiring decisions. The project constructs simulated interactions for its collaborative experiments. That is a reasonable starting point for development, but the resulting evaluation must be interpreted as a controlled comparison on synthetic behaviour. The distinction between a public posting corpus and artificial interaction histories is essential to understanding the strength of the findings.

Chapter 5: Recommendation Methodology

5.1 From a profile to a ranked list

The recommendation process begins with a user profile and ends with an ordered set of jobs. Each candidate is represented by normalised skills, free-text experience and a desired role. Each job has a corresponding text representation and a list of annotated skills. The content component can compare these immediately. When interaction history is available, a collaborative component generates an additional score for each catalogue item. The hybrid then combines the two vectors, applies any requested restrictions and returns the highest-ranked eligible records. This design separates scoring from the display of results, allowing the same recommender object to support the website and the JSON API.

The mathematical objective is ranking, not salary prediction or classification into “hire” and “reject” categories. Every posting receives a relative relevance score derived from available signals. The score's value depends on preprocessing, score normalisation and the other jobs in the collection. It should therefore be understood as a ranking instrument. For example, a displayed match score of 0.73 does not mean that a candidate has a 73% chance of receiving an interview or satisfying all of a job's requirements.

5.2 Building the content representation

The data-loading module constructs a textual document for each job by concatenating three copies of the title, two copies of the annotated skills and one copy of the description. Repetition gives the short, informative fields more influence during vectorisation. The *TfidfVectorizer* uses unigrams and bigrams, an English stop-word list, a maximum of 40,000 features, a minimum document frequency of two and sublinear term-frequency scaling. Bigrams are useful because an expression such as “data analyst” can carry more specific meaning than either word independently.

In simplified notation, the TF-IDF weight for a term is the product of a term-frequency component and an inverse document-frequency component. The exact implementation uses scikit-learn's configured weighting and normalisation rather than a hand-written formula. Once a posting is transformed into a vector, the user profile is passed through the same fitted vocabulary. Profile text repeats the desired title and skills three times before adding the experience summary. This deliberate weighting increases the role of information supplied explicitly in the profile.

Textual cosine similarity compares the profile vector with every job vector. A second similarity is calculated from binary skill-set vectors. For a set of user skills U and job skills J, the binary cosine is proportional to the number of shared skills divided by the geometric mean of the two skill-set sizes. This calculation rewards overlap but also reflects list lengths. Where the user has recognised skills in the fitted vocabulary, the model calculates *content = 0.5 × text cosine + 0.5*

× *skill cosine*. If the skill vector is empty, it falls back to text similarity alone. Thus, the model does not entirely fail when a user's spelling or terminology produces no recognised explicit skills.

5.3 Preparing collaborative evidence

The collaborative system represents behaviour in a sparse matrix whose rows correspond to users and columns correspond to jobs. A recorded view contributes a weight of 1, a save 3 and an application action 5. Multiple interaction types for the same pair accumulate before a *log(1 + weight)* transformation dampens unusually large values. The purpose is to avoid allowing the most active observations to overwhelm the similarity calculations. The interactions used in the packaged demonstration are predominantly simulated; separately, the Flask app adds actual local interactions recorded during use of the prototype.

The matrix is sparse because a typical user interacts with only a small fraction of the 8,000-item catalogue. This fact is central to recommendation: a system is expected to rank thousands of unseen opportunities based on a short observed history. JobRec constructs an item-to-item similarity matrix by computing cosine relationships among the job columns. A user's existing interactions can then be propagated across similar items to estimate which other postings might be of interest.

5.4 Neighbourhood and latent-factor signals

The neighbourhood score is obtained by multiplying a user's interaction vector by the item-similarity matrix. If the user previously engaged with jobs that are commonly explored together by other users, those shared patterns increase the estimated relevance of unvisited jobs. A separate TruncatedSVD transformation captures a 40-dimensional latent representation, subject to the number of available users and matrix dimensions. This technique compresses the large interaction space and may reflect broader patterns of common interest that are not visible through a single job pair.

The code projects a user vector into the item-factor space and reconstructs scores for the catalogue. Negative reconstructed values are clipped to zero. Both the neighbourhood scores and latent scores are then independently scaled by their maximum positive values, and the final collaborative score is their average. The system sets scores to zero for items the user has already interacted with in the collaborative component. In the online recommendation flow, previously applied jobs are also explicitly excluded. These details distinguish the implemented model from a generic description of collaborative filtering.

Combining neighbourhood and latent methods can reduce dependence on one signal, but it also introduces computational and interpretative costs. The item-similarity matrix has dimensions associated with an 8,000-job catalogue and can become expensive as the number of items grows. The stored model is therefore suitable for a compact academic demonstration; more scalable

retrieval and approximate-nearest-neighbour methods would merit investigation if the catalogue expanded substantially.

5.5 Hybrid fusion, normalisation and cold start

The implementation normalises the content vector by dividing it by its maximum positive value before blending. The collaborative vector has already been normalised within its own component. For users with a usable collaborative vector, the final calculation is $hybrid = \alpha \times normalised\ content + (1 - \alpha) \times collaborative$, with α set to 0.3 in the packaged model. The content and collaborative values are combined across the whole catalogue before location, remote and salary restrictions are applied. The highest eligible items are then selected.

Cold start requires a specific rule rather than just an adjustment to alpha. When no usable collaborative signal exists, the code returns the original content score and labels the mode "content-only (cold start)." This avoids ranking all jobs equally for someone who has not clicked or saved anything. It also creates a transparent behavioural difference: two users with similar stated skills may initially receive similar content recommendations, but their results can diverge after interactions are recorded.

The code supports updating only the collaborative component. The web application collects its SQLite interaction records, combines them with the packaged mock interactions and calls a collaborative refit after every five new interaction-recording events, or after a manual retraining request. This is a simple adaptation mechanism rather than continuous online gradient learning. It would need more sophisticated scheduling and cache management in a heavily used deployment.

5.6 Restrictions and explanations

After calculating scores, the system can apply a case-insensitive location filter and a Boolean remote-only filter. A minimum-salary setting compares the entered amount against the available annual salary field. There is an important implementation issue here: jobs with missing salary values are treated as having an infinite salary for the comparison, so they are not excluded solely because the field is missing. The present filter is better interpreted as excluding known salaries below the threshold, not validating that every recommended job meets a verified minimum.

For each recommendation, the model attaches the required skills that appear in the user's profile and those that do not. The interface displays the former in green and the latter in a neutral style. It also shows the overall hybrid score and component scores. These explanations are deliberately concrete, focusing on what the candidate might know or need to learn. However, the displayed list is a comparison against extracted skill labels, not a full assessment of competence, seniority or credential requirements. A missing tag does not necessarily mean that the user lacks the knowledge; it may simply not have been entered into the profile.

5.7 Reproducible training and model storage

The training script loads the curated dataset, fits the content representation, constructs simulated users and interaction records, and then fits the complete hybrid object. The model is serialised to *models/hybrid_recommender.pkl*, while user profiles and interactions are preserved in a JSON file. The archive already includes both outputs, allowing the web demonstration to load the model without repeating every training step. Source scripts remain available so that the data-processing and model-building stages can be inspected or rerun.

Chapter 6: System Architecture and Implementation

6.1 Overall architecture

JobRec has three principal layers: a data and model layer, a Flask application layer and a browser interface. The data layer contains the curated CSV, the fitted TF-IDF representation, the interaction model and the saved evaluation outputs. The application layer receives requests, checks login state where necessary, reads or updates SQLite records and invokes recommendation methods. The presentation layer renders templates that organise profile information, search results, job descriptions and recommendation explanations into pages. Keeping these responsibilities reasonably separate makes the system understandable and permits changes to one layer without rewriting every other component.

At startup, the Flask application loads the saved hybrid model and the mock-interaction data. It derives a ranked skill vocabulary for the profile form and initialises SQLite tables if they do not exist. Model loading at startup avoids retraining on every page request. The trade-off is that startup depends on the presence of a valid, compatible saved model; if that file is missing, the application exits with an instruction to run the training script.

6.2 Database design and persistence

SQLite stores two main logical entities. The *users* table holds an account identifier, username, email, hashed password, name, education, skills, experience, preferred title, location preference, remote flag, salary preference and creation time. The *interactions* table records the user identifier, job identifier, interaction type and timestamp. A uniqueness constraint covers each combination of user, job and interaction type. This design allows a job to be viewed and saved by the same user without creating unlimited duplicate records of either type.

Passwords are hashed with Werkzeug before storage and verified with the corresponding checker at login. Flask sessions identify authenticated users. The login-required decorator protects profile, recommendation, saved-history and interaction endpoints. These mechanisms are appropriate foundations for a small prototype. However, the database is a local file and no role-

based administration, verified email workflow or secure hosted deployment is evidenced in the archive.

6.3 Profile and user flow

The landing page introduces the purpose of the application, shows dataset statistics and explains how recommendations are generated. A new visitor can browse listings before registering. Registration collects basic account details; the user then visits the profile page, where skills, experience and desired role become inputs to the recommendation engine. The page presents common skills as selectable suggestions, which reduces the friction of manually entering technology names and encourages the use of vocabulary recognised in the dataset.

After saving the profile, the user is directed to ranked recommendations. The visible example includes a data-science-oriented profile with SQL, Python, Excel, Tableau, Power BI and machine learning. Each job card shows a position and company, employment details, skill overlap and a numerical score, followed by actions for detail viewing, saving and applying. This sequence brings the most decision-relevant information to the top of the page without requiring the candidate to read every full description first.

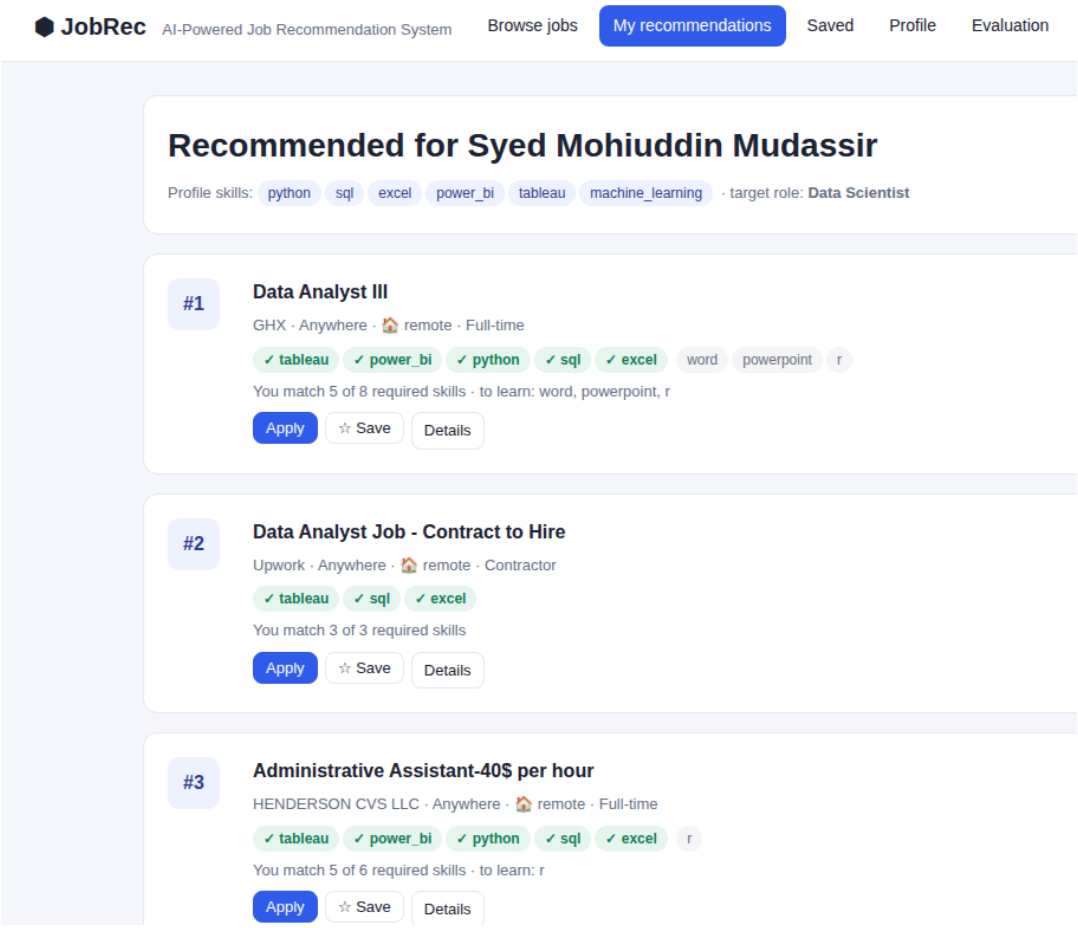


Figure 1. Cropped JobRec interface showing personalised ranking and skill explanations.

6.4 Browse, details and history

The browse page provides a conventional search baseline. It filters job titles, annotated skill strings and descriptions using a case-insensitive literal text match and supports optional location and remote flags. The results are limited to the first 50 matching records shown on that page, while the page can display the overall number of matches. This is not the same technique as the recommendation model. It is a straightforward search facility that provides a familiar route through the data and a visible comparison with personalised ranking.

The detail page displays the posting's description and several similar jobs calculated through content-vector cosine similarity. For authenticated users, opening the detail page creates a view interaction. Save and Apply are POST actions, both recorded locally. The saved-history page lists the user's saved and applied records with timestamps. Because the same vacancy can produce different interaction types, the history reflects a small progression of interest rather than a single yes/no marker.

6.5 Evaluation page and REST interface

The evaluation page reads saved JSON results and displays comparisons for popularity, content-only, collaborative and hybrid recommendation. It also embeds the stored precision-and-recall chart and shows how the hybrid coefficient affects the results. This is useful in an academic demonstration because the interface exposes the empirical basis of the design choice rather than hiding evaluation in a separate notebook.

6.6 Technologies and practical trade-offs

The project uses Python for modelling and server logic, pandas for tabular data, NumPy and SciPy for numerical and sparse-matrix operations, scikit-learn for TF-IDF, cosine similarity, K-Means and TruncatedSVD, Flask for routing, Werkzeug for password hashing, SQLite for persistence and Matplotlib for evaluation plots. HTML templates and CSS define the user interface. This stack is accessible to students and enables end-to-end investigation without depending on a large cloud infrastructure.

That accessibility comes with limitations. Local SQLite is simple but not ideal for substantial concurrent writes. The application maintains a global in-memory recommender and refits collaborative filtering within the web process. This may interrupt responsiveness when model training becomes expensive. An operational system would typically separate background model updates, database access and request serving more carefully. For a bounded 8,000-item demonstration, however, the compact design makes it possible to inspect how an interaction affects future rankings.

Chapter 7: Experimental Design and Evaluation Protocol

7.1 Purpose and evaluation questions

The evaluation compares recommendation methods under a common dataset and synthetic interaction environment. Its immediate purpose is to ask whether a hybrid of content and collaborative scores retrieves more held-out interactions near the top of a ranked list than either component alone. A second question concerns the sensitivity of the hybrid to alpha, the coefficient controlling the blend. A third concerns the usual precision–recall trade-off as the displayed list expands from five to ten or twenty items. The study reports results for these questions rather than making claims about applicant outcomes or job-offer success.

The supplied results were generated offline and saved as structured JSON, a Markdown summary and a plot. The reported numbers can therefore be read directly from the project artefacts. Although the code supports rerunning the evaluation, the present report analyses those saved outputs and checks their consistency with the evaluation source. This avoids presenting an additional, undocumented experiment as though it were part of the submitted project.

7.2 Synthetic users and interaction generation

The project creates 300 mock user profiles. To give these profiles a realistic degree of internal structure, it first uses K-Means clustering with 25 clusters on job text vectors. The cluster labels are described as career tracks. For each track, the simulator counts common skills and forms a short list of prominent technologies. A mock user is assigned to a track, receives several sampled skills and may acquire one additional noise skill. A seed posting from the track supplies a plausible desired title, while an automatically generated experience statement refers to years of work and selected tools.

Each mock user then interacts with roughly eight to twenty jobs drawn from the chosen cluster, with sampling probability influenced by squared skill overlap. The interaction type is labelled as view, save or apply according to fixed random probabilities. This creates structured relevance: users with similar career tracks tend to interact with overlapping parts of the catalogue. The generation routine uses fixed seeds, supporting repeatable experiments. It also embeds a methodological limitation: similarity between profile skills and job skills influences the creation of the interactions that later serve as the evaluation target.

Consequently, the evaluation is not an unbiased measurement of actual human preferences. It is a useful synthetic stress test for components designed to recover structured patterns. In particular, collaborative filtering can benefit when mock users share tracks and the test events originate from the same general interaction-generating process as the training events. Results should be interpreted as performance on this controlled task, not as evidence that the same hit rates would be achieved with job seekers behaving independently in the real world.

7.3 Train/test partitioning and item exclusion

The simulator produces 4,213 interaction records in total. The evaluation routine shuffles each user's records using a fixed seed and places approximately 30% of that user's events into a test set, ensuring that at least one interaction is held out. The reported split contains 3,068 training interactions and 1,145 held-out test interactions. Unlike a naive global split, this procedure gives every user some history for collaborative training while retaining individual unseen interactions for evaluation.

The collaborative model is fitted on training interactions only. For each user, the evaluator constructs content, collaborative, hybrid and popularity scores across the catalogue. Jobs observed in the user's training history are removed from consideration before selecting top-k lists. A recommendation counts as correct if its job identifier appears among that user's held-out interactions. This exclusion rule is essential; otherwise, recommending something already known to be in the user's history could inflate the apparent quality of the system.

The evaluation uses a random per-user split, not a chronological future-prediction split. Such a split measures recovery of withheld observations rather than the ability to predict what a candidate will consider next month. If temporal generalisation is important, future work should train on earlier events and test on later events. It would also be appropriate to ensure that any parameter tuning is carried out using a validation set separate from the data reserved for the final report.

7.4 Baseline and comparison models

Four configurations form the principal comparison. The popularity baseline ranks jobs by the number of training interactions and ignores the current user's detailed profile. The content model uses TF-IDF and explicit skill similarity. The collaborative model uses the combined neighbourhood and TruncatedSVD interaction scores. The hybrid blends normalised content and collaborative signals at the selected coefficient. All methods are evaluated using the same catalogue, user set, held-out interactions and displayed list sizes.

The popularity baseline is useful because it represents a simple, non-personalised strategy. It asks whether personalisation improves on presenting advertisements that receive the most engagement overall. Content and collaborative methods then act as ablations of the hybrid: removing one source of information reveals how much the combined approach gains under the protocol. The alpha sweep extends this comparison by showing whether the hybrid benefits from a balanced mix or relies mainly on one component.

7.5 Definitions of evaluation measures

Precision@k is calculated as the number of held-out relevant jobs found in the first k recommendations divided by k. Recall@k divides the same number of hits by the total number of held-out relevant jobs for that user. Hit-rate@k is one if at least one relevant job occurs in the first k positions and zero otherwise; the reported hit rate is the average across users. NDCG@k gives higher credit to relevant recommendations occurring early in the ranking using a logarithmic discount and then divides by the best possible discounted gain for the user's available relevant items.

For example, if a user has four held-out relevant jobs and two appear in the first ten recommendations, precision@10 equals 2/10 = 0.20 and recall@10 equals 2/4 = 0.50. That user contributes a hit-rate value of 1. The NDCG value also depends on whether those two hits occur in positions one and two or positions nine and ten. In this way, the four metrics answer related but non-identical questions about recommendation quality. Averaging across the 300 users produces the values reported in the experimental tables.

7.6 Methodological safeguards and reservations

The comparison has several desirable controls: the catalogue is shared across models, the split uses fixed randomness, the test items are excluded from each user's training set and the same metrics are applied throughout. These features support an internal comparison. Nevertheless, several design choices limit the broader interpretation. The profiles and interactions are jointly simulated from the posting corpus; no external relevance labels are available; and the candidate alpha values are assessed against the same held-out set used to communicate the final result.

That last issue should be addressed explicitly. Choosing alpha = 0.3 because it gives the best precision@10 on the reported test set can produce selection bias. A stronger study would set aside a validation subset for hyperparameter choice and reserve an independent test set for the final figures. It would also run multiple seeds, report confidence intervals and compare results across different simulated user populations. The current values are useful descriptive findings, but they should not be reported as statistically conclusive evidence of superiority in live recruitment.

Chapter 8: Results and Interpretation

8.1 Overall comparison at k = 10

The recorded evaluation shows a consistent ordering across the four methods at ten recommendations. The popularity baseline obtains precision@10 of 0.011, recall@10 of 0.026, hit-rate@10 of 0.077 and NDCG@10 of 0.020. Content-only ranking increases these measures to

0.018, 0.050, 0.137 and 0.038. Collaborative filtering is substantially stronger under the synthetic protocol, scoring 0.076, 0.194, 0.430 and 0.154. The hybrid achieves the highest central values: precision@10 = 0.089, recall@10 = 0.226, hit-rate@10 = 0.480 and NDCG@10 = 0.182.

Table 3. Baseline and hybrid results at k = 10 (synthetic evaluation)

Model	Precision	Recall	Hit rate	NDCG
Popularity	0.011	0.026	0.077	0.020
Content	0.018	0.050	0.137	0.038
Collaborative	0.076	0.194	0.430	0.154
Hybrid	**0.089**	**0.226**	**0.480**	**0.182**

These numbers mean that the hybrid finds, on average, approximately 0.89 held-out relevant jobs within each ten-item recommendation list, not that 8.9 of ten recommendations are useful. A hit rate of 0.480 means that 48% of the 300 mock users have at least one held-out relevant item in the top ten. Recall@10 of 0.226 means that about 22.6% of their held-out relevant interactions are retrieved in those first ten suggestions, averaged across users. Because users generally have several held-out interactions, precision and hit rate naturally describe different aspects of the same ranked output.

8.2 Results at five, ten and twenty suggestions

At k=5, the hybrid reaches precision 0.118, recall 0.150, hit rate 0.390 and NDCG 0.147. At k=20, its precision decreases to about 0.063 while recall rises to 0.317, hit rate to 0.580 and NDCG to 0.218. This is a familiar ranking trade-off: a longer list captures more potentially relevant items but reduces the average concentration of relevant items within the visible set. The same general pattern is apparent in the collaborative and content baselines.

Table 4. Hybrid performance as list length changes

k	Precision	Recall	Hit rate	NDCG
5	0.118	0.150	0.390	0.147
10	0.089	0.226	0.480	0.182
20	0.063	0.317	0.580	0.218

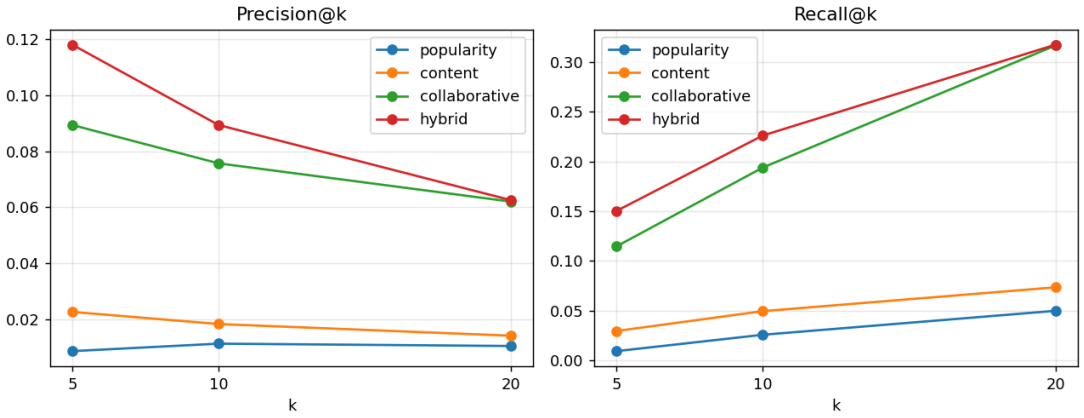


Figure 2. Precision@k and recall@k across popularity, content, collaborative and hybrid scoring.

The choice of list length should therefore reflect the interface's purpose. A short, carefully ranked list supports quick exploration. A larger list may be appropriate when candidates want to survey alternatives more broadly, but it increases reading effort and the chance of repetitive or weak matches. The application supports selecting the number of displayed jobs. In future user research, it would be useful to test not only retrieval measures but also how often candidates actually inspect recommendations beyond the first few positions.

8.3 Hybrid weighting experiment

The alpha sweep provides evidence about the relative importance of the two information sources in the simulated environment. With alpha = 0, the system reduces to the collaborative component and precision@10 is 0.076. Increasing alpha to 0.1 raises precision to 0.081; at 0.2 it reaches 0.085; and at 0.3 it reaches 0.089. Beyond that point, the reported top-ten precision declines, reaching 0.083 at 0.4, 0.071 at 0.5 and 0.018 for content alone at alpha = 1.0. The NDCG@10 curve follows a similar pattern, with the recorded maximum of 0.182 at alpha = 0.3.

Table 5. Hybrid coefficient sensitivity at k = 10

α	**Precision**	**Recall**	**NDCG**
0.0	0.076	0.194	0.154
0.1	0.081	0.206	0.164
0.2	0.085	0.213	0.173
0.3	**0.089**	**0.226**	**0.182**
0.4	0.083	0.210	0.177
0.5	0.071	0.182	0.156
0.6	0.056	0.144	0.128
0.8	0.028	0.072	0.067
1.0	0.018	0.050	0.038

This does not mean that “30% content and 70% collaboration” is a universally optimal recipe for job recommendations. The result reflects the current data, scoring normalisation and synthetic behaviour generator. In fact, at k=20 some other alpha choices produce slightly higher recall than alpha = 0.3. The choice is defensible for a top-ten-focused prototype, but a deployed system would need to revisit the coefficient after collecting real, consented behaviour and deciding which user-facing objective matters most.

8.4 Interpretation of the displayed recommendations

The recommendations interface offers a concrete example of the system's behaviour. A profile emphasising Python, SQL, Excel, Power BI and Tableau receives a ranked set of analytics-related postings and nearby roles. The interface identifies matching skills, shows missing requirements such as R or other tools, and provides a score and component values. This demonstrates that the ranking is presented in a way users can inspect. However, it also illustrates a limitation: some high-ranked roles may not match a person's intended occupation closely merely because they share software requirements or collaborative patterns.

8.5 What the experiment establishes

Within the reported synthetic experiment, the hybrid offers better top-ten ranking measures than the three named comparison systems. The advantage is visible in precision, recall, hit rate and NDCG, rather than only in a single selected indicator. This is a useful outcome for the main research question. It shows that the implemented fusion can recover more withheld interactions than the separate models under the specified assumptions.

The experiment does not establish that the recommendation scores are calibrated probabilities, that the vacancies remain available, that candidates would prefer the suggested jobs, or that employers would accept them. There is also no record of independent human relevance assessment or a controlled deployment. The most appropriate conclusion is consequently specific: the project demonstrates a functioning hybrid job recommender whose saved offline evaluation improves on its baselines in a 300-user simulated setting. That statement is both positive and appropriately limited.

Chapter 9: Testing, Ethics and Critical Discussion

9.1 Implementation evidence and test coverage

The code and screenshots support a structured walk-through of key application functions: visiting the home page, creating a profile, generating ranked suggestions, searching jobs, viewing descriptions, saving or applying, and opening the evaluation page. The presence of routes and templates provides evidence that these features were implemented. The archived screenshots

show plausible outputs, including component scores, matched skills and saved histories. The saved evaluation files support numerical comparison between the algorithms. Together these sources constitute a stronger basis for reporting than interface screenshots alone.

Nevertheless, the ZIP does not provide a comprehensive automated test suite, documented performance load tests or signed user-acceptance results. The report therefore does not claim that every edge case has been verified in an independent test campaign. A sensible acceptance plan would exercise empty profiles, unrecognised skill names, duplicate saves, invalid job identifiers, extreme top-k values, empty filter results and attempts to access protected routes without login. It would also compare the API results with the corresponding web view and test whether retraining changes rankings without corrupting the SQLite interaction history.

9.2 Data protection and interaction meaning

The application processes account data and records behaviour, both of which can reveal employment interests. For a classroom demonstration, the built-in mock profiles avoid the need to collect sensitive histories from real users. If the system were offered publicly, however, people would need clear notice about the purposes of collection, retention and model training. Secure secret management, database backups, user consent and the ability to correct or delete profile data would become essential parts of the engineering requirements.

The words used for actions also matter. Selecting Apply in this application records the action inside JobRec, but the code does not show a request to an employer's application endpoint. Describing that event as an externally completed job application would be misleading. In a future version, a separate “Visit employer application” action, along with a confirmation of external submission where technically possible, would make the distinction clearer. Accurate interface terminology is part of responsible software design.

9.3 Fairness, transparency and feedback loops

The model does not appear to use protected demographic characteristics as explicit ranking features. That alone is insufficient to establish fairness. Job descriptions, locations and previous interaction histories can reflect unequal opportunity, and a system that learns from popularity or collaborative patterns can amplify established preferences. The present dataset is heavily centred on analytics tools and US-based postings, so its coverage should not be mistaken for a neutral view of all occupations or regions.

There is also a feedback-loop risk. Candidates are more likely to interact with jobs the system places near the top, and those interactions may then reinforce the same ranking tendencies during collaborative updates. A future evaluation should examine job and employer exposure, diversity across occupational categories, geographic accessibility and outcomes for different kinds of

profiles. Explainability would improve if the interface distinguished a strong skill-based fit from a high collaborative score driven mainly by other users' activity.

9.4 Practical limitations

Several limitations arise from the available implementation rather than from recommender theory in general. The posting catalogue is static and historical. Salary coverage is low, and missing values are not filtered as a cautious candidate might expect. The content model relies largely on surface word features rather than deeper semantic representations. Synthetic interactions are generated from clustered job content, making the evaluation less independent of its input representations than genuine human behaviour would be. The hybrid weight is selected within the same evaluation environment, and no variance estimates are reported.

The system also lacks evidence of scalable model serving, production-grade authentication policy, accessibility auditing and direct employer integration. These observations do not invalidate the prototype; they establish the work needed to move from a convincing demonstration to an operational service. Clear reporting of such limitations is important because an academic project should not treat all future development tasks as accomplishments of the current version.

Chapter 10: Recommendations for Further Development

10.1 Data and relevance improvements

The most valuable next step would be to evaluate the recommender with consented participants rather than only simulated profiles. Participants could judge whether suggested jobs are interesting, feasible and appropriate for their experience, allowing relevance to be measured independently of the synthetic interaction generator. A validation subset could be used for choosing alpha and other model settings, followed by an untouched test set. Multiple seeds and uncertainty intervals would make performance comparisons more informative.

Data quality should be improved in parallel. An updated job feed with timestamps and expiry checks would address the static-catalogue problem. Salary filters should distinguish unknown salary from confirmed salary above the threshold and let users decide whether to include unknown values. More complete skill extraction and a consistent vocabulary would reduce missed matches. Adding seniority, qualifications, industry, contract terms and workplace location at greater granularity would permit eligibility filters beyond textual similarity alone.

10.2 Modelling and interface improvements

Future modelling could compare the current TF-IDF representation with sentence embeddings that capture more contextual similarity between different expressions. Such a comparison should

retain the classical approach as a transparent baseline rather than assuming a newer method must perform better. Candidate-interest diversity, duplicate suppression and career-level compatibility could also be incorporated into the final ranking stage. If the user opts in, feedback controls such as “not relevant” might provide a more direct training signal than a view alone.

For the interface, useful additions include a clearer explanation of why a recommendation appeared, a warning when salary information is missing, and direct links to verified employer application pages. The current matched/missing skill display should be retained because it supports learning and self-assessment. From an infrastructure perspective, model refreshes should run outside the request process, with logging, versioning, automated tests, stronger secret handling and monitored response times. Those changes would turn the experimental architecture into a more maintainable service.

Chapter 11: Conclusion

The JobRec project demonstrates how skills information and behavioural evidence can be brought together in a browser-based job recommendation prototype. Its content component represents job titles, annotated skills and descriptions using TF-IDF and cosine similarity. Its collaborative component interprets weighted interactions through item similarity and TruncatedSVD latent factors. A hybrid ranking combines these approaches, supports a content-only path for cold-start users and presents matched and missing skills to make recommendations more understandable.

The available dataset contains 8,000 public job postings and 127 distinct skill labels. The recorded offline experiment evaluates four ranking methods with 300 simulated users and 4,213 interaction records. At ten suggestions, the hybrid achieves precision 0.089, recall 0.226, hit rate 0.480 and NDCG 0.182, improving on the included popularity, content and collaborative baselines. These findings support the effectiveness of the implemented blend within the evaluation's defined conditions.

The principal limitation is the gap between a controlled synthetic test and real-world employment matching. User behaviour was simulated, job data are historical, and no external evidence of hiring outcomes is available. The contribution should therefore be understood as a researched, implemented and evaluated prototype rather than a proven recruitment decision system. With stronger data freshness, independent relevance judgements, careful security design and wider user testing, the same architecture could provide a foundation for further study of reliable and explainable job recommendations.

References

Adomavicius, G., & Tuzhilin, A. (2005). Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. IEEE Transactions on Knowledge and Data Engineering, 17(6), 734–749. https://doi.org/10.1109/TKDE.2005.99

Barousse, L. (n.d.). Data Analyst Job Postings [Pay, Skills, Benefits] [Data set]. Kaggle. https://www.kaggle.com/datasets/lukebarousse/data-analyst-job-postings-google-search

Burke, R. (2002). Hybrid recommender systems: Survey and experiments. User Modeling and User-Adapted Interaction, 12(4), 331–370. https://doi.org/10.1023/A:1021240730564

Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative filtering for implicit feedback datasets. In 2008 Eighth IEEE International Conference on Data Mining (pp. 263–272). IEEE. https://doi.org/10.1109/ICDM.2008.22

Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. ACM Transactions on Information Systems, 20(4), 422–446. https://doi.org/10.1145/582415.582418

Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. Computer, 42(8), 30–37. https://doi.org/10.1109/MC.2009.263

Paparrizos, I., Cambazoglu, B. B., & Gionis, A. (2011). Machine learned job recommendation. In Proceedings of the Fifth ACM Conference on Recommender Systems (pp. 325–328). Association for Computing Machinery. https://doi.org/10.1145/2043932.2043994

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://www.jmlr.org/papers/v12/pedregosa11a.html

Resnick, P., & Varian, H. R. (1997). Recommender systems. Communications of the ACM, 40(3), 56–58. https://doi.org/10.1145/245108.245121

Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2022). Recommender systems handbook (3rd ed.). Springer. https://doi.org/10.1007/978-1-0716-2197-4

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0

Appendix A: Source Artefacts and Traceability

This appendix identifies the files used as the primary evidence for project-specific facts in the report. The files are retained in the project archive. External literature supports the technical concepts; the project files establish what was actually implemented and recorded.

Table A1. Project artefacts used to verify the implementation

File or location	Evidence
data/jobs_dataset.csv	8,000 records, column schema, 127 skill labels, missing salary values
data/prepare_dataset.py	Documented data-cleaning and sampling procedure
jobrec/content_model.py	TF-IDF parameters and explicit skill cosine score

File or location	Evidence
jobrec/collab_model.py	View/save/apply weights, item similarity and TruncatedSVD
jobrec/hybrid.py	Fusion, filters, cold-start mode and explanations
jobrec/simulate.py	Synthetic user profiles, tracks and interaction generation
jobrec/evaluate.py	Per-user holdout protocol and top-k metric formulas
app/app.py and templates	Routes, SQLite schema, account flows and interface
outputs/evaluation_results.json	Numerical model comparison and alpha sweep
outputs/screenshots/	Reference images of the implemented interface